

Thesis Title: AI Driven Flood susceptibility Analysis in south Punjab Using Remote sensing and GIS techniques

Contents

Thesis Title: AI Driven Flood susceptibility Analysis in south Punjab Using Remote sensing and GIS techniques	1
Python code for Flood susceptibility	2
Random Forest Classification Code JavaScript.....	17
Drainage Density	22
Soil type.....	24
Rainfall.....	25
Multi-Year Flood Progression Map.....	27
Danger Zone / Risk zone.....	31

Python code for Flood susceptibility

```
import ee
import geemap
import geopandas as gpd
import math
import time
import os

# Initialize Earth Engine with error handling
def authenticate_and_initialize():
    try:
        ee.Initialize()
        print("Earth Engine initialized!")
    except Exception as e:
        print(f"Initial initialization failed: {e}")
        try:
            ee.Authenticate(auth_mode='notebook', force=True)
            ee.Initialize()
            print("Earth Engine initialized after re-authentication!")
        except Exception as e:
            print(f"Authentication failed: {e}")
            print("Please grant Earth Engine and Drive permissions in the browser. See  

http://goo.gle/ee-auth." )
            exit()

# Check Drive access
def check_drive_access():
    try:
        test_image = ee.Image.constant(0).clip(ee.Geometry.Point([0, 0]).buffer(100))
        task = ee.batch.Export.image.toDrive(
```

```

        image=test_image,
        description='test_drive_access',
        folder='test',
        scale=30,
        maxPixels=1e6
    )
    task.start()
    print("✓ Drive access verified")
    return True
except Exception as e:
    print(f'Drive access test failed: {e}')
    print("Ensure your Google account has Drive access and re-authenticate.")
    return False

# Authenticate and initialize
authenticate_and_initialize()
if not check_drive_access():
    exit()

# 1. Load Muzaffargarh Boundary
muzaffargarh_shp = r"D:\Thesis\Shapefile\Muzaffrahgarh\Muzaffrahgarh.shp"
try:
    gdf = gpd.read_file(muzaffargarh_shp)
    muzaffargarh_geom = geemap.geopandas_to_ee(gdf)
    muzaffargarh_geom = muzaffargarh_geom.geometry().simplify(100)
    print("✓ Muzaffargarh boundary loaded and converted to Geometry")

# Get bounding box for reference
bounds = muzaffargarh_geom.bounds().getInfo()['coordinates'][0]

```

```

    print(f'✓ Study area bounds: {bounds}')
except Exception as e:
    print(f'Error loading shapefile: {e}')
    exit()

# Define common projection and scale
COMMON_SCALE = 30
COMMON_CRS = 'EPSG:4326'

# Function to resample and reproject to common specifications
def standardize_raster(image, geometry):
    return image.reproject(crs=COMMON_CRS, scale=COMMON_SCALE).clip(geometry)

# 2. Load and Process Data for 2025 Projections

# DEM (SRTM 30m) - static dataset (no change expected)
try:
    dem = ee.Image('USGS/SRTMGL1_003').select('elevation')
    dem = standardize_raster(dem, muzaffargarh_geom)
    slope = ee.Terrain.slope(dem)
    slope = standardize_raster(slope, muzaffargarh_geom)
    print("✓ DEM and Slope loaded for Muzaffargarh (static for 2025)")
except Exception as e:
    print(f'Error loading DEM: {e}')
    exit()

# LULC 2025 Projection (based on urbanization and agricultural expansion)
try:
    # Use 2022 LULC as baseline and project to 2025

```

```

lulc_2022 = ee.ImageCollection('ESA/WorldCover/v200').first().select('Map')

# Identify urban areas (class 50) and agricultural areas (class 40)
urban_areas_2022 = lulc_2022.eq(50)
agri_areas_2022 = lulc_2022.eq(40)

# Project urban expansion (3% annual growth for Muzaffargarh)
urban_expansion = urban_areas_2022.focal_max(radius=400, units='meters') # Urban sprawl

# Project agricultural expansion into marginal lands
agri_expansion = agri_areas_2022.focal_max(radius=300, units='meters')

# Create 2025 LULC projection
lulc_2025 = lulc_2022.where(urban_expansion.gt(0.4), 50) # Convert to urban
lulc_2025 = lulc_2025.where(agri_expansion.gt(0.3).And(lulc_2022.neq(50)), 40) # Convert
to agriculture

lulc = standardize_raster(lulc_2025, muzaffargarh_geom)
print("✓ LULC 2025 projection created (urban & agricultural expansion)")
except Exception as e:
    print(f'Error creating LULC 2025 projection: {e}')
    exit()

# NDVI 2025 Projection (based on climate change and land use trends)
try:
    # Use 2022 NDVI as baseline
    ndvi_2022 = ee.ImageCollection('COPERNICUS/S2_SR_HARMONIZED') \
        .filterBounds(muzaffargarh_geom) \
        .filterDate('2022-04-01', '2022-05-31') \
        .filter(ee.Filter.lt('CLOUDY_PIXEL_PERCENTAGE', 30)) \

```

```

.median() \
.normalizedDifference(['B8', 'B4'])

# Projected NDVI for 2025 considering:
# - Climate stress: -2% per year
# - Urbanization impact: Additional -5% in urban areas
ndvi_2025 = ndvi_2022.multiply(0.94) # -6% over 3 years

# Further reduce in urban areas
urban_mask = lulc.eq(50)
ndvi_2025 = ndvi_2025.where(urban_mask, ndvi_2025.multiply(0.95))

ndvi = standardize_raster(ndvi_2025, muzaffargarh_geom)
print("✓ NDVI 2025 projection created (climate & urbanization impacts)")
except Exception as e:
    print(f'Error computing NDVI 2025 projection: {e}')
    exit()

# Rainfall 2025 Projection (climate change scenarios for Punjab)
try:
    # Use 2022 as baseline (extreme flood year)
    rainfall_2022 = ee.ImageCollection('UCSB-CHG/CHIRPS/DAILY') \
        .filterBounds(muzaffargarh_geom) \
        .filterDate('2022-06-01', '2022-09-30') \
        .sum()

    # Projected rainfall for 2025 based on IPCC AR6 projections:
    # - Increased monsoon intensity in South Asia: +3-5% per decade
    # - More extreme rainfall events

```

```

rainfall_2025 = rainfall_2022.multiply(1.08) # +8% increase by 2025 from 2022 baseline

rainfall = standardize_raster(rainfall_2025, muzaffargarh_geom)
print("✓ Rainfall 2025 projection created (+8% climate change scenario)")
except Exception as e:
    print(f'Error loading Rainfall 2025 projection: {e}')
    exit()

# Drainage Density 2025 (considering land use changes)
try:
    # Calculate flow accumulation
    filled_dem = ee.Algorithms.TerrainFill(dem, 50)
    flow_dir = ee.Terrain.flowDirection(filled_dem)
    flow_acc = ee.Terrain.flowAccumulation(flow_dir, 1)

    # Adjust stream threshold for urbanization (reduced infiltration)
    urban_mask = lulc.eq(50)
    # Lower threshold in urban areas, higher in natural areas
    stream_threshold = flow_acc.gt(1000).where(urban_mask, flow_acc.gt(600))

    # Calculate drainage density
    drainage_density = stream_threshold.focal_mean(radius=150, units='meters',
kernelType='circle')
    drainage_density = standardize_raster(drainage_density, muzaffargarh_geom)
    print("✓ Drainage density 2025 projection calculated (urbanization impact)")
except Exception as e:
    print(f'Error computing Drainage Density: {e}')
    # Create a fallback drainage density
    print("Creating fallback drainage density...")
    drainage_density = slope.multiply(0.03).min(1.0) # Higher for urbanized future

```

```

drainage_density = standardize_raster(drainage_density, muzaffargarh_geom)
print("✓ Fallback drainage density created")

# 3. Susceptibility Layers (5-class scoring system) for 2025 Projection
print("Creating 2025 susceptibility layers with 5-class system...")

# Elevation susceptibility for 2025 (unchanged physically)
dem_suscept = ee.Image(1) \
    .where(dem.gt(200), 0.2) \
    .where(dem.gt(150), 0.4) \
    .where(dem.gt(100), 0.6) \
    .where(dem.gt(50), 0.8) \
    .where(dem.lte(50), 1.0)

# Slope susceptibility (unchanged)
slope_suscept = ee.Image(1) \
    .where(slope.gt(10), 0.2) \
    .where(slope.gt(5), 0.4) \
    .where(slope.gt(2), 0.6) \
    .where(slope.gt(1), 0.8) \
    .where(slope.lte(1), 1.0)

# LULC susceptibility for 2025 (updated for projected changes)
lulc_suscept = lulc.remap(
    [10, 20, 30, 40, 50, 60, 70, 80, 90, 95, 100],
    [0.2, 0.4, 0.3, 0.85, 0.9, 0.7, 0.1, 1.0, 0.95, 0.85, 0.3],
    0.5
)

```



```
# NDVI susceptibility for 2025 (lower NDVI in urban areas = higher susceptibility)
```

```
ndvi_suscept = ee.Image(1) \  
  .where(ndvi.gt(0.6), 0.2) \  
  .where(ndvi.gt(0.4), 0.4) \  
  .where(ndvi.gt(0.2), 0.6) \  
  .where(ndvi.gt(0.0), 0.8) \  
  .where(ndvi.lte(0.0), 1.0)
```

```
# Rainfall susceptibility for 2025 (higher thresholds for increased rainfall)
```

```
rainfall_suscept = ee.Image(1) \  
  .where(rainfall.lt(350), 0.2) \  
  .where(rainfall.lt(550), 0.4) \  
  .where(rainfall.lt(750), 0.6) \  
  .where(rainfall.lt(950), 0.8) \  
  .where(rainfall.gte(950), 1.0)
```

```
# Drainage density susceptibility for 2025
```

```
drainage_suscept = ee.Image(1) \  
  .where(drainage_density.lt(0.2), 0.2) \  
  .where(drainage_density.lt(0.4), 0.4) \  
  .where(drainage_density.lt(0.6), 0.6) \  
  .where(drainage_density.lt(0.8), 0.8) \  
  .where(drainage_density.gte(0.8), 1.0)
```

```
print("✓ All 2025 susceptibility layers created with 5-class system")
```

```
# 4. Weighted Susceptibility for 2025 (future climate and land use considerations)
```

```
weights = {  
  'dem': 0.12,    # Slight increase due to sea level rise concerns
```

```

'slope': 0.13, # Reduced as urbanization alters natural slopes
'lulc': 0.25, # Increased weight due to significant land use changes
'ndvi': 0.10, # Climate stress indicator
'rainfall': 0.30, # High weight for climate change impacts
'drainage': 0.10 # Modified by urbanization
}

```

```

susceptibility = dem_suscept.multiply(weights['dem']) \
.add(slope_suscept.multiply(weights['slope'])) \
.add(lulc_suscept.multiply(weights['lulc'])) \
.add(ndvi_suscept.multiply(weights['ndvi'])) \
.add(rainfall_suscept.multiply(weights['rainfall'])) \
.add(drainage_suscept.multiply(weights['drainage'])) \
.clip(muzaaffargarh_geom)

```

```

print("✓ 2025 Weighted susceptibility calculated")

```

5. Classify Susceptibility into 5 classes for 2025

```

susceptibility_classified = ee.Image(1) \
.where(susceptibility.lte(0.2), 1) \
.where(susceptibility.lte(0.4), 2) \
.where(susceptibility.lte(0.6), 3) \
.where(susceptibility.lte(0.8), 4) \
.where(susceptibility.gt(0.8), 5) \
.clip(muzaaffargarh_geom) \
.rename('flood_susceptibility_2025')

```

```

print("✓ 2025 Classified susceptibility (1-5: Very Low to Very High)")

```

```

# 6. Classify all rasters into 5 classes for consistency
def classify_5_classes(image, thresholds, class_values=[1, 2, 3, 4, 5]):
    """Classify image into 5 classes based on thresholds"""
    classified = ee.Image(class_values[0])
    for i, threshold in enumerate(thresholds):
        classified = classified.where(image.gt(threshold), class_values[i+1])
    return classified

# Classify individual 2025 projected rasters
dem_classified = classify_5_classes(dem, [50, 100, 150, 200])
slope_classified = classify_5_classes(slope, [1, 2, 5, 10])
ndvi_classified = classify_5_classes(ndvi, [0.0, 0.2, 0.4, 0.6])
rainfall_classified = classify_5_classes(rainfall, [350, 550, 750, 950])
drainage_classified = classify_5_classes(drainage_density, [0.2, 0.4, 0.6, 0.8])

print("✓ All 2025 projected rasters classified into 5 classes")

# 7. Export 2025 Projection Rasters to Google Drive
export_folder = 'Muzaffargarh_2025_Projection_5Class'
export_params = {
    'region': muzaffargarh_geom,
    'scale': COMMON_SCALE,
    'crs': COMMON_CRS,
    'maxPixels': 1e10,
    'folder': export_folder,
    'fileFormat': 'GeoTIFF'
}

# List of 2025 projected rasters to export

```

```

rasters = [
    (dem_classified, 'DEM_5Class_2025'),
    (slope_classified, 'Slope_5Class_2025'),
    (lulc, 'LULC_2025_Projection'),
    (ndvi_classified, 'NDVI_5Class_2025'),
    (rainfall_classified, 'Rainfall_5Class_2025'),
    (drainage_classified, 'Drainage_Density_5Class_2025'),
    (susceptibility_classified, 'Flood_Susceptibility_5Class_2025'),
    (susceptibility, 'Flood_Susceptibility_Continuous_2025')
]

```

```

# Start export tasks with status monitoring

```

```

tasks = []

```

```

for raster, name in rasters:

```

```

    try:

```

```

        task = ee.batch.Export.image.toDrive(
            image=raster,
            description=name,
            **export_params
        )

```

```

        task.start()

```

```

        tasks.append((task, name))

```

```

        print(f"✓ Export task started for {name}")

```

```

        time.sleep(2)

```

```

    except Exception as e:

```

```

        print(f"Error starting export for {name}: {e}")

```

```

# Monitor task status

```

```

print("\nMonitoring export tasks...")

```

```

completed_tasks = 0
for task, name in tasks:
    try:
        status = task.status()
        while status['state'] in ['READY', 'RUNNING']:
            print(f"Task {name}: {status['state']}")
            time.sleep(10)
            status = task.status()
        if status['state'] == 'COMPLETED':
            print(f"✓ Task {name} completed successfully")
            completed_tasks += 1
        else:
            print(f"Task {name} failed with state: {status['state']}")
            if 'error_message' in status:
                print(f"Error: {status['error_message']}")
    except Exception as e:
        print(f"Error monitoring task {name}: {e}")

print(f"\n{completed_tasks}/{len(tasks)} export tasks completed successfully.")
print("Check Google Drive folder '{}' for GeoTIFF files.".format(export_folder))

```

8. Visualization for 2025 Projections

```
Map = geemap.Map(center=[30.0, 71.0], zoom=9) # Centered on Muzaffargarh
```

```
# Add 2025 projection layers
```

```
try:
```

```
    Map.addLayer(muzaffargarh_geom, {'color': 'black'}, 'Muzaffargarh Boundary', False)
```

```
    Map.addLayer(dem_classified, {'min': 1, 'max': 5, 'palette': ['darkblue', 'blue', 'green', 'yellow', 'red']}, 'DEM 2025 (5 Class)', False)
```

```
Map.addLayer(slope_classified, {'min': 1, 'max': 5, 'palette': ['darkgreen', 'green', 'yellow', 'orange', 'red']}, 'Slope 2025 (5 Class)', False)
```

```
# LULC 2025 with specific visualization
```

```
lulc_vis = {'min': 10, 'max': 100, 'palette': ['006400', '90EE90', 'FFFF00', '8B4513', '0000FF', 'FF0000', '808080', 'FFFFFF', 'A9A9A9', '000080', '00FFFF']}
```

```
Map.addLayer(lulc, lulc_vis, 'LULC 2025 Projection', False)
```

```
Map.addLayer(ndvi_classified, {'min': 1, 'max': 5, 'palette': ['brown', 'yellow', 'lightgreen', 'green', 'darkgreen']}, 'NDVI 2025 (5 Class)', False)
```

```
Map.addLayer(rainfall_classified, {'min': 1, 'max': 5, 'palette': ['white', 'lightblue', 'blue', 'darkblue', 'purple']}, 'Rainfall 2025 (5 Class)', False)
```

```
Map.addLayer(drainage_classified, {'min': 1, 'max': 5, 'palette': ['white', 'lightcyan', 'cyan', 'blue', 'darkblue']}, 'Drainage Density 2025 (5 Class)', False)
```

```
# 2025 Flood susceptibility with 5-class color scheme
```

```
Map.addLayer(susceptibility_classified,
              {'min': 1, 'max': 5, 'palette': ['#00FF00', '#ADFF2F', '#FFFF00', '#FFA500', '#FF0000']},
              'Flood Susceptibility 2025 Projection (5 Class)', True)
```

```
# Add comprehensive legend for 2025
```

```
Map.add_layer(title='5-Class Flood Susceptibility 2025',
              legend_dict={
                  'Very Low (1)': '#00FF00',
                  'Low (2)': '#ADFF2F',
                  'Moderate (3)': '#FFFF00',
                  'High (4)': '#FFA500',
                  'Very High (5)': '#FF0000'
              }, position='bottomright')
```

```
Map.addLayerControl()
```

```

print("✓ Map created with all 2025 projection layers")

except Exception as e:
    print(f"Error adding layers to map: {e}")

# Display 2025 projection results
print("\n" + "="*70)
print("FLOOD SUSCEPTIBILITY ANALYSIS - MUZAFFARGARH 2025 PROJECTION")
print("="*70)
print("Region: Muzaffargarh District, Punjab, Pakistan")
print("Period: 2025 Projection (Climate Change & Urbanization Scenarios)")
print("Resolution: 30m | CRS: EPSG:4326")
print("\nPROJECTION ASSUMPTIONS:")
print(" • LULC Changes:")
print("   - Urban expansion: 3% annual growth")
print("   - Agricultural expansion into marginal lands")
print("   - Reduced natural vegetation cover")
print(" • Climate Change Impacts:")
print("   - Rainfall: +8% increase from 2022 baseline")
print("   - NDVI: -6% overall reduction (climate stress)")
print("   - Additional -5% NDVI reduction in urban areas")
print(" • Hydrological Changes:")
print("   - Reduced infiltration in urban areas")
print("   - Altered drainage patterns")
print("\nPARAMETERS & WEIGHTS (adjusted for future conditions):")
for param, weight in weights.items():
    print(f" {param.upper():<8}: {weight:.2f}")
print("\nCLIMATE CHANGE CONTEXT (IPCC AR6):")
print(" • South Asia: Increased monsoon intensity")

```

```

print(" • More frequent extreme rainfall events")
print(" • Higher flood risks in Indus River basin")
print("\nURBANIZATION TRENDS IN MUZAFFARGARH:")
print(" • Rapid urban expansion along highways")
print(" • Conversion of agricultural land to urban")
print(" • Increased impervious surfaces")
print("\nSUSCEPTIBILITY CLASSES:")
print(" 1: Very Low | 2: Low | 3: Moderate | 4: High | 5: Very High")
print("\nANALYSIS PURPOSE:")
print(" • Long-term flood risk assessment")
print(" • Climate adaptation planning")
print(" • Urban development guidelines")
print(" • Agricultural resilience strategies")
print("="*70)

```

Show map

Map

change year according to your choice

Random Forest Classification Code JavaScript

```
// =====  
  
// 1. Study Area  
  
// =====  
  
var studyArea = table.merge(table2);  
Map.centerObject(studyArea, 9);  
Map.addLayer(studyArea, {color:'red', fillColor:'00000000'}, 'Study Area');  
  
// =====  
  
// 2. Sentinel-2 composite + indices  
  
// =====  
  
function getComposite(year, cloudMax) {  
  var col = ee.ImageCollection('COPERNICUS/S2_SR_HARMONIZED')  
    .filterBounds(studyArea)  
    .filterDate(year+'-01-01', year+'-12-31')  
    .filter(ee.Filter.lt('CLOUDY_PIXEL_PERCENTAGE', cloudMax))  
    .select(['B2','B3','B4','B5','B6','B7','B8','B8A','B11','B12']);  
  
  var median = col.median().clip(studyArea);  
  
  var ndvi = median.normalizedDifference(['B8','B4']).rename('NDVI');  
  var ndwi = median.normalizedDifference(['B3','B8']).rename('NDWI');  
  var ndbi = median.normalizedDifference(['B11','B8']).rename('NDBI');  
  
  return median.addBands([ndvi, ndwi, ndbi]);  
}  
  
// =====  
  
// 3. Composites
```

```

// =====
var comp2019 = getComposite('2019', 20);
var comp2022 = getComposite('2022', 20);
var comp2025 = getComposite('2025', 30);

// =====
// 4. Prepare training points
// =====
function prepareTraining(collection, classValue) {
  return collection.map(function(f) {
    return f.set('Class', classValue);
  });
}

var trainingPointsNumeric = prepareTraining(Barren, 1)
  .merge(prepareTraining(Vegetation, 2))
  .merge(prepareTraining(Water, 3))
  .merge(prepareTraining(Builtup, 4));

Map.addLayer(trainingPointsNumeric, {color:'black'}, 'Training Points');

// =====
// 5. Bands
// =====
var bands = ['B2','B3','B4','B5','B6','B7','B8','B8A','B11','B12','NDVI','NDWI','NDBI'];

// =====
// 6. Color palette
// =====

```

```

var classPalette = ['#d3d3d3', '#00ff00', '#0000ff', '#ff0000']; // Barren, Vegetation, Water, Built-
up

// =====
// 7. Classification function
// =====
function classifyYear(image, year) {

  // Sample image at training points
  var samples = image.sampleRegions({
    collection: trainingPointsNumeric,
    properties: ['Class'],
    scale: 10,
    geometries: true
  });

  // Split into train (70%) / test (30%)
  var split = samples.randomColumn('random');
  var train = split.filter(ee.Filter.lt('random', 0.7));
  var test = split.filter(ee.Filter.gte('random', 0.7));

  // Train Random Forest
  var classifier = ee.Classifier.smileRandomForest(100).train({
    features: train,
    classProperty: 'Class',
    inputProperties: bands
  });

  // Classify image
  var classified = image.select(bands).classify(classifier);

```

```

// Accuracy assessment
var testPred = test.classify(classifier);
var errorMatrix = testPred.errorMatrix('Class', 'classification');
print('--- Accuracy report ' + year + ' ---');
print('Error Matrix:', errorMatrix);
print('Overall Accuracy:', errorMatrix.accuracy());
print('Kappa:', errorMatrix.kappa());

// Display classified image
Map.addLayer(classified,
  {min:1, max:4, palette:classPalette},
  'LULC ' + year
);

// Return classified image for export
return classified;
}

// =====
// 8. Classify all years
// =====
var classified2019 = classifyYear(comp2019, '2019');
var classified2022 = classifyYear(comp2022, '2022');
var classified2025 = classifyYear(comp2025, '2025');

// =====
// 9. Export each year to Drive
// =====
var exports = [

```

```

    {img: classified2019, year: '2019'},
    {img: classified2022, year: '2022'},
    {img: classified2025, year: '2025'}
  ];

  exports.forEach(function(e) {
    Export.image.toDrive({
      image: e.img,
      description: 'LULC_' + e.year,
      folder: 'LULC_Maps',
      fileNamePrefix: 'LULC_' + e.year,
      region: studyArea,
      scale: 10,
      maxPixels: 1e13
    });
  });
});

// =====
// 10. Add legend
// =====

var legend = ui.Panel({style: {position: 'bottom-left', padding: '8px 15px'}});
legend.add(ui.Label('LULC Classes'));

var classNames = ['Barren', 'Vegetation', 'Water', 'Built-up'];
for (var i = 0; i < classNames.length; i++) {
  var colorBox = ui.Label("", {backgroundColor: classPalette[i], padding: '8px', margin: '2px 0'});
  var nameLabel = ui.Label(classNames[i], {margin: '2px 0 2px 6px'});
  legend.add(ui.Panel([colorBox, nameLabel], ui.Panel.Layout.Flow('horizontal')));
}
Map.add(legend);

```

Change your area and year

Drainage Density

```
// 1 Load AOI
var AOI = ee.FeatureCollection("projects/ee-msarfrazakhtar9944/assets/Rajanpur");

// 2 Load MERIT Hydro
var hydro = ee.Image('MERIT/Hydro/v1_0_1');
var upa = hydro.select('upa');

// 3 Define streams
var threshold_km2 = 0.1; // lower to ensure raster visibility
var streams = upa.gt(threshold_km2).updateMask(upa.gt(threshold_km2));

// 4 Convert streams raster to vector polygons
var streamsVector = streams.reduceToVectors({
  geometry: AOI,
  scale: 90,
  geometryType: 'polygon',
  eightConnected: true
});

// 5 Add a numeric property to each feature for rasterization
streamsVector = streamsVector.map(function(f){
  return f.set('value', 1);
});

// 6 Convert vector back to raster
```

```

var streamRasterFromVector = streamsVector.reduceToImage({
  properties: ['value'],    // numeric property to rasterize
  reducer: ee.Reducer.first() // take first (value = 1)
});

// 7 Compute drainage density per pixel (km/km²)
var pixelSize_km = 0.09;
var pixelArea_km2 = pixelSize_km * pixelSize_km;
var drainageDensityRaster =
streamRasterFromVector.multiply(pixelSize_km).divide(pixelArea_km2);

// 8 Smooth raster with 3×3 kernel
var kernel = ee.Kernel.square({radius: 1, units: 'pixels', normalize: true});
drainageDensityRaster = drainageDensityRaster.convolve(kernel);

// 9 Clip to AOI
drainageDensityRaster = drainageDensityRaster.clip(AOI);

// 10 Visualization
var visParams = {min:0, max:20, palette:['white','lightblue','blue','darkblue']};
Map.centerObject(AOI, 10);
Map.addLayer(drainageDensityRaster, visParams, 'Smoothed Drainage Density');
Map.addLayer(AOI, {}, 'AOI boundary');

// 1 1 Export raster as GeoTIFF
Export.image.toDrive({
  image: drainageDensityRaster,
  description: 'Muzaffrahgarh_DrainageDensity_Smooth',
  scale: 90,

```

```

    region: AOI,
    fileFormat: 'GeoTIFF',
    maxPixels: 1e13
  });

```

Soil type

```
// SOIL CLAY MAP FOR RAJANPUR DISTRICT
```

```
var rajanpur = ee.FeatureCollection("projects/ee-msarfrazakhtar9944/assets/Rajanpur");
```

```
// Get soil clay data (0-30cm average)
```

```

var soilClay = ee.Image("projects/soilgrids-isric/clay_mean")
  .select(['clay_0-5cm_mean', 'clay_5-15cm_mean', 'clay_15-30cm_mean'])
  .reduce(ee.Reducer.mean())
  .clip(rajanpur)
  .rename('clay_percentage');

```

```
// Add to map for preview
```

```

Map.centerObject(rajanpur, 9);
Map.addLayer(rajanpur, {color: 'red', fillColor: '00000000'}, 'Rajanpur District');
Map.addLayer(soilClay, {
  min: 10,
  max: 60,
  palette: ['#ffffcc', '#c7e9b4', '#7fcdbb', '#41b6c4', '#2c7fb8', '#253494', '#081d58']
}, 'Soil Clay Content (%)');

```



```
// Export for ArcGIS
Export.image.toDrive({
  image: soilClay,
  description: 'Rajanpur_Soil_Clay_Map',
  scale: 250,
  region: rajanpur.geometry(),
  maxPixels: 1e9,
  folder: 'GEE_Exports',
  fileNamePrefix: 'rajanpur_soil_clay',
  crs: 'EPSG:4326',
  fileFormat: 'GeoTIFF'
});

print('✅ Rajanpur soil clay map ready for export! Check Tasks tab.');
```

Rainfall

```
// Load your Muzaffargarh region
var muzaffargarh = ee.FeatureCollection("projects/ee-msarfrazakhtar9944/assets/Rajanpur");

// Calculate total rainfall June-September 2025
var totalRainfall = ee.ImageCollection('UCSB-CHG/CHIRPS/DAILY')
  .filterDate('2019-06-01', '2019-09-30')
  .filterBounds(muzaffargarh)
  .select('precipitation')
  .sum()
  .clip(muzaffargarh);

// Preview on map
```

```

Map.centerObject(muzaffargarh, 9);
Map.addLayer(muzaffargarh, {color: 'red'}, 'Muzaffargarh');
Map.addLayer(totalRainfall, {min: 0, max: 400, palette: ['white', 'blue', 'darkblue']}, 'Rainfall');

// EXPORT 1: Main rainfall raster
Export.image.toDrive({
  image: totalRainfall,
  description: 'Rajanpur_Rainfall_2019',
  scale: 1000,
  region: muzaffargarh.geometry(),
  maxPixels: 1e9,
  folder: 'GEE_Exports',
  fileNamePrefix: 'rainfall_Rajanpur_2019',
  crs: 'EPSG:4326',
  fileFormat: 'GeoTIFF'
});

// EXPORT 2: District boundary
Export.table.toDrive({
  collection: muzaffargarh,
  description: 'Muzaffargarh_Boundary',
  folder: 'GEE_Exports',
  fileNamePrefix: 'muzaffargarh_boundary',
  fileFormat: 'SHP'
});

print('✅ Ready to export! Check Tasks tab.');
```

```

print('1. Total Rainfall Raster will be exported as GeoTIFF');
```

```

print('2. District Boundary will be exported as Shapefile');
```

Multi-Year Flood Progression Map

// **1** LOAD DISTRICTS

```
var muzaffargarh = ee.FeatureCollection('projects/ee-  
msarfrazakhtar9944/assets/Muzaffrahgarh');  
  
var multan = ee.FeatureCollection('projects/ee-msarfrazakhtar9944/assets/Multan');  
var rajanpur = ee.FeatureCollection('projects/ee-msarfrazakhtar9944/assets/Rajanpur');  
var dgkhan = ee.FeatureCollection('projects/ee-msarfrazakhtar9944/assets/DG_khan');
```

// Merge all districts

```
var roi = muzaffargarh.merge(multan).merge(rajanpur).merge(dgkhan);  
Map.centerObject(roi, 9);  
Map.addLayer(roi, {color: 'black', fillColor: '00000000'}, 'District Boundaries');
```

// 2. CLOUD MASK FUNCTION (Sentinel-2 SR)

```
function maskS2Clouds(image) {  
  var qa = image.select('QA60');  
  var cloudBitMask = 1 << 10;  
  var cirrusBitMask = 1 << 11;  
  var mask = qa.bitwiseAnd(cloudBitMask).eq(0)  
    .and(qa.bitwiseAnd(cirrusBitMask).eq(0));  
  return image.updateMask(mask)  
    .divide(10000)  
    .copyProperties(image, ['system:time_start']);  
}
```

// 3. DEFINE PERMANENT WATER (PRE-FLOOD 2025 BASELINE)

```

var beforeCol = ee.ImageCollection('COPERNICUS/S2_SR_HARMONIZED')
  .filterBounds(roi)
  .filterDate('2025-01-01', '2025-02-28')
  .filter(ee.Filter.lt('CLOUDY_PIXEL_PERCENTAGE', 20))
  .map(maskS2Clouds);

var beforeImg = beforeCol.median();
var mndwiBefore = beforeImg.normalizedDifference(['B3', 'B11']).rename('MNDWI_Before');
var permanentWater = mndwiBefore.gt(0); // This is now defined before use

```

// 4. HELPER FUNCTION FOR YEARLY FLOODS

```

// Assigns a unique integer to each year to create a single categorical layer
function getFloodYear(startDate, endDate, permWater, yearValue) {
  var col = ee.ImageCollection('COPERNICUS/S2_SR_HARMONIZED')
    .filterBounds(roi)
    .filterDate(startDate, endDate)
    .filter(ee.Filter.lt('CLOUDY_PIXEL_PERCENTAGE', 20))
    .map(maskS2Clouds);

  var floodImg = col.median();
  var mndwi = floodImg.normalizedDifference(['B3', 'B11']);
  var floodWater = mndwi.gt(0);

  // Keep only water that IS NOT permanent water, then multiply by yearValue
  return floodWater.where(permWater, 0).selfMask().multiply(yearValue).toByte();
}

```

// 5. CALCULATE FLOOD LAYERS

```

// Values: 2=2019, 3=2022, 4=2025
var flood2019 = getFloodYear('2019-07-01', '2019-09-30', permanentWater, 2);

```

```

var flood2022 = getFloodYear('2022-07-01', '2022-09-30', permanentWater, 3);
var flood2025 = getFloodYear('2025-07-01', '2025-09-30', permanentWater, 4);

// 6. COMBINE INTO SINGLE LAYER
// Starts with Permanent Water (Value 1) and overlays newer years on top
var multiYearFlood = ee.Image(1).updateMask(permanentWater)
  .blend(flood2019)
  .blend(flood2022)
  .blend(flood2025)
  .clip(roi);

// 7. VISUALIZATION & MAPPING
var multiPalette = [
  '#08306b', // 1: Permanent Water (Dark Blue)
  '#9ecae1', // 2: 2019 Flood (Light Blue)
  '#4292c6', // 3: 2022 Flood (Medium Blue)
  '#ef3b2c' // 4: 2025 Flood (Red)
];

Map.addLayer(multiYearFlood, {min: 1, max: 4, palette: multiPalette}, 'Flood Progression
(2019-2025)');

// 8. TITLE & LEGEND UI
var title = ui.Panel({
  style: {position: 'top-center', padding: '8px'}
});
title.add(ui.Label({
  value: 'Multi-Year Flood Progression Map',
  style: {fontSize: '18px', fontWeight: 'bold'}
}));

```

```

Map.add(title);

var legend = ui.Panel({
  style: {position: 'bottom-left', padding: '10px'}
});

function legendItem(color, label) {
  return ui.Panel([
    ui.Label({style: {backgroundColor: color, padding: '8px', border: '1px solid black'}}),
    ui.Label({value: label, style: {margin: '0 0 0 6px'}})
  ], ui.Panel.Layout.Flow('horizontal'));
}

legend.add(ui.Label('Legend', {fontWeight: 'bold'}));
legend.add(legendItem('#1F4E79', 'Permanent Water'));
legend.add(legendItem('#FFE699', 'Flood 2019 Only'));
legend.add(legendItem('#F4A300', 'Flood 2022 Only'));
legend.add(legendItem('#C00000', 'Flood 2025 Only'));
Map.add(legend);

// 9. EXPORT COMBINED LAYER
Export.image.toDrive({
  image: multiYearFlood,
  description: 'Muzaffargarh_MultiYear_Flood_2019_2025',
  folder: 'GEE_Exports',
  region: roi,
  scale: 10,
  maxPixels: 1e13
});

```

Danger Zone / Risk zone

```
// =====  
  
// LOAD DISTRICT  
  
// =====  
  
var district = ee.FeatureCollection(  
  'projects/ee-msarfrazakhtar9944/assets/Rajanpur'  
);  
  
Map.centerObject(district, 9);  
Map.addLayer(district, {color: 'black'}, 'Rajanpur');  
  
// =====  
  
// LOAD RIVERS  
  
// =====  
  
var river = ee.FeatureCollection(  
  'projects/ee-msarfrazakhtar9944/assets/pakistan-waterways-shape'  
);  
  
// =====  
  
// CREATE MULTI DISTANCE BUFFERS (meters)  
  
// =====  
  
var bufferDistances = [1000, 3000, 5000, 7000, 10000]; // in meters  
var buffers = bufferDistances.map(function(dist){  
  return river.map(function(f){  
    return f.buffer(dist);  
  }).geometry().intersection(district.geometry());  
});
```

```

// =====
// RISK RASTER
// =====

var base = ee.Image(0).clip(district);

var risk = base

  .where(ee.Image().paint(buffers[4], 1), 1) // Very Low
  .where(ee.Image().paint(buffers[3], 2), 2) // Low
  .where(ee.Image().paint(buffers[2], 3), 3) // Moderate
  .where(ee.Image().paint(buffers[1], 4), 4) // High
  .where(ee.Image().paint(buffers[0], 5), 5); // Very High

// =====
// DISPLAY RISK ZONES
// =====

var palette = [
  '#313695', // Very Low (Oxford Atlas style)
  '#74add1', // Low
  '#fee08b', // Moderate
  '#f46d43', // High
  '#a50026' // Very High
];

Map.addLayer(risk, {min: 1, max: 5, palette: palette}, 'River Risk Zones');

// =====
// ADD LEGEND
// =====

var legend = ui.Panel({style: {position: 'bottom-right', padding: '8px 15px'}});

```



```

// Legend title
var legendTitle = ui.Label({
  value: 'River Flood Risk Zones',
  style: {fontWeight: 'bold', fontSize: '14px', margin: '0 0 6px 0', padding: '0'}
});
legend.add(legendTitle);

// Risk labels and colors
var riskLevels = ['Very Low', 'Low', 'Moderate', 'High', 'Very High'];
for (var i = 0; i < riskLevels.length; i++) {
  var colorBox = ui.Label({
    style: {
      backgroundColor: palette[i],
      padding: '8px',
      margin: '0 0 4px 0'
    }
  });
  var description = ui.Label({
    value: riskLevels[i],
    style: {margin: '0 0 4px 6px'}
  });

  var legendItem = ui.Panel({
    widgets: [colorBox, description],
    layout: ui.Panel.Layout.Flow('horizontal')
  });

  legend.add(legendItem);
}

```

```
Map.add(legend);
Export.image.toDrive({
  image: risk,
  description: 'Muzaffargarh_River_Risk_Zones',
  folder: 'GEE_Exports',      // Change folder name if you want
  fileNamePrefix: 'Muzaffargarh_Risk',
  region: district.geometry(), // Clip to district boundary
  scale: 30,                  // Set pixel resolution (meters)
  crs: 'EPSG:4326',           // WGS84, change if needed
  maxPixels: 1e13
});
```

You can use all these codes for your study. The first susceptibility code is only in python all other codes are Google Earth Engine codes GEE code written in JavaScript.

Change your range of study your Area of interest and your required year and I hope you will get benefit from all these codes.

Best Regards;

Muhammad Sarfraz

BS-SSR01 F22

Session (2022-2026)

For more info

Contact # +92-3097338102

Email: msarfrazakhtar99@gmail.com